

Mini Project – Smart Asset Tracking System

Project Overview

In this mini project, students will build a modern **Asset Tracking System** using:

- C#
- .NET Console Application
- Entity Framework Core
- SQL Database
- LINQ
- Object-Oriented Programming (OOP)
- CRUD Operations
- Reporting Features

The application will help companies track physical assets such as:

- Laptop computers
- Desktop computers
- Mobile phones
- Tablets
- Office equipment

The system should support:

- Database storage using Entity Framework Core
 - Asset lifecycle tracking
 - Office management
 - Currency conversion
 - Reporting and filtering
 - Clean architecture and reusable code
-

General Requirements

Asset Rules

Each asset should contain:

- Id
- Asset Type
- Brand
- Model Name
- Purchase Date
- Purchase Price (USD)
- Local Price (To be converted)
- Office Location
- Serial Number
- Employee/Username (optional)
- Warranty Expiration Date

Asset Lifecycle

For this project:

- Asset lifetime = 3 years
- YELLOW => warning = less than 3 months remaining
- RED => warning = less than 6 months remaining

Technical Requirements

Students should use:

- Entity Framework Core
- SQL Server or SQLite
- Migrations
- LINQ queries
- Repository or Service structure if possible
- Exception handling
- Validation for incorrect inputs

LEVEL 1 — Basic Asset Registration

Goal

Create a console application that can store company assets in a database.

Requirements

Create appropriate classes for:

Computer Assets

- Laptop
- Desktop Computer

Mobile Assets

- iPhone
- Samsung
- Nokia
- Tablet

Required Features

- Create assets
- Save assets to database
- Read and display all assets
- Use Entity Framework Core
- Create proper constructors and properties
- Use inheritance where appropriate

Suggested Class Structure

- Asset (Base Class)
- ComputerAsset
- MobileAsset

Example Console Output

===== ASSET LIST =====				
ID	Type	Brand	Model	Purchase Date

1	Laptop	Lenovo	ThinkPad X1	2024-02-11
2	Mobile	Apple	iPhone 15	2025-01-10
3	Desktop	Asus	ROG Station	2023-09-15
=====				

LEVEL 2 — Full CRUD and Asset Status

Goal

Expand the application with full asset management.

New Requirements

Your application should support FULL CRUD:

- Create
- Read
- Update
- Delete

Additional Features

- Sort assets by category first
- Then sort by purchase date
- Highlight assets based on remaining lifetime

Asset Status Rules

Remaining Lifetime	Status
Less than 3 months	YELLOW
Less than 6 months	RED
More than 6 months	NORMAL

Suggested Improvements

- Use LINQ for sorting and filtering
- Create reusable methods
- Add menu-driven navigation
- Add validation for invalid dates or empty fields

Example Console Output

```
===== COMPANY ASSETS =====

Computers
-----
ID   Brand   Model           Purchase Date   Status
-----
1    Lenovo  ThinkPad X1     2023-08-01     YELLOW
2    Asus    ZenBook 14      2022-06-10     RED

Mobile Devices
-----
3    Apple   iPhone 15       2025-01-10     NORMAL
4    Samsung Galaxy S24    2024-03-05     NORMAL

=====
```

Example Menu

```
1. Add Asset
2. Show All Assets
3. Update Asset
4. Delete Asset
5. Search Asset
6. Exit
```

Type	Brand	Model	Office	Purchase Date	Price in USD	Currency	Local price today
Phone	iPhone	8	Spain	12/29/2018	970	EUR	801.65
Computer	HP	Elitebook	Spain	6/1/2019	1423	EUR	1176.03
Phone	iPhone	11	Spain	9/25/2020	990	EUR	818.18
Phone	iPhone	X	Sweden	7/15/2018	1245	SEK	10375
Phone	Motorola	Razr	Sweden	3/16/2020	970	SEK	8083.33
Computer	HP	Elitebook	Sweden	10/2/2020	588	SEK	4900
Computer	Asus	W234	USA	4/21/2017	1200	USD	1200
Computer	Lenovo	Yoga 730	USA	5/28/2018	835	USD	835
Computer	Lenovo	Yoga 530	USA	5/21/2019	1030	USD	1030

LEVEL 3 — Global Office Management

Goal

Add support for multiple company offices around the world.

Requirements

Create offices such as:

- Sweden Office
- USA Office
- Germany Office
- Turkey Office

Each office should contain:

- Office Name
- Country
- Currency

Currency Support

Assets are entered in USD.

The application should:

- Convert prices into local currency
- Display currency symbols correctly
- Use exchange rates

NOTE: For educational purposes, students may either:

- Use fixed conversion rates manually
- OR connect to a real exchange-rate API

Reporting Features

Add reporting functionality such as:

- Total asset value per office
- Asset count per office
- Assets close to expiration
- Most expensive assets

Example Console Output

```
===== STOCKHOLM OFFICE =====

ID  Asset      Brand   Price      Status
-----
1   Laptop    Lenovo  15,500 SEK  NORMAL
2   Mobile    Apple   8,200 SEK  YELLOW

Total Office Value: 23,700 SEK

=====
```

Example Report Output

```
===== REPORT =====

Office Asset Counts
-----
Sweden Office : 12
USA Office    : 18
Germany Office: 9
Turkey Office : 15

Assets Near Expiration
-----
Lenovo ThinkPad X1
Asus ZenBook
Nokia XR20


↓


=====
```


LEVEL 4 — Advanced Search, Filtering, and Export

Goal

Make the application more professional and business oriented.

New Requirements

Add advanced features such as:

Search Features

- Search by brand
- Search by model
- Search by office
- Search by purchase year

Filtering Features

- Show only expired assets
- Show only computers
- Show only mobile devices
- Show assets by office

Export Features

Export reports to:

- TXT file
- CSV file
- JSON file

Additional Suggestions

- Use async database operations
- Add pagination for large lists
- Add logging
- Use configuration files

Example Console Output

```
===== SEARCH RESULT =====

Search: Brand = Lenovo

ID  Type      Model      Office
-----
1   Laptop    ThinkPad X1  Sweden
7   Laptop    Legion 5    Germany

=====
```

Example CSV Export

```
Id,Type,Brand,Model,Office,Price
1,Laptop,Lenovo,ThinkPad X1,Sweden,15500
2,Mobile,Apple,iPhone 15,USA,1200
```

LEVEL 5 — Enterprise Asset Management System (Advanced)

Goal

Transform the project into a small enterprise-ready system.

Advanced Requirements

User Roles

Add different system roles:

- Admin
- Manager
- Employee

Authentication Simulation

Implement a simple login system:

```
Username: admin  
Password: 1234
```

Asset Assignment

Assets can be assigned to employees.

Each employee should contain:

- Employee Id
- Full Name
- Department
- Email
- Assigned Assets

Maintenance Tracking

Track:

- Last maintenance date
- Next maintenance date
- Maintenance notes

Dashboard Statistics

Display:

- Total assets
- Total office value
- Expiring assets
- Assets per employee
- Most used asset type

Bonus Ideas (Optional)

- Build REST API
- Add unit tests
- Create Blazor or ASP.NET MVC frontend
- Add Docker support
- Add role-based authorization

Example Console Output

```
===== DASHBOARD =====

Logged in as: Admin

Total Assets           : 125
Total Employees        : 42
Expiring Soon          : 8
Total Company Value    : 2,450,000 SEK

Top Asset Type         : Laptop
Most Expensive Office  : Stockholm

=====
```

Example Employee Assignment Output

```
===== EMPLOYEE ASSETS =====

Employee : Sarah Johnson
Department: IT Support

Assigned Assets
-----
1. Lenovo ThinkPad X1
2. iPhone 15 Pro
3. Samsung Tablet S9

=====
```

Suggested Folder Structure

```
/Models  
/Services  
/Data  
/Repositories  
/Helpers  
/Reports
```

Recommended Technologies

Beginner Friendly

- .NET 8,9,10
- MS Server
- Entity Framework Core

More Advanced

- SQL Server
 - ASP.NET Core Web API
 - AutoMapper
 - Dependency Injection
 - Serilog
-

Important Notes

- Use meaningful naming conventions
 - Avoid duplicated code
 - Keep methods small and reusable
 - Use comments where necessary
 - Follow clean coding principles
 - Test the application carefully
 - Handle invalid user inputs gracefully
-

Challenge Ideas (Optional)

Students who finish early may additionally:

- Create a graphical UI
 - Add barcode support
 - Generate PDF reports
 - Add email notifications
 - Connect to cloud database
 - Build REST API endpoints
 - Add unit and integration tests
-

Final Goal

By completing this project, students should practice:

- Real-world software structure
- Database management
- Entity Framework Core
- Object-oriented programming
- LINQ
- Reporting systems
- Console application architecture
- Professional coding practices